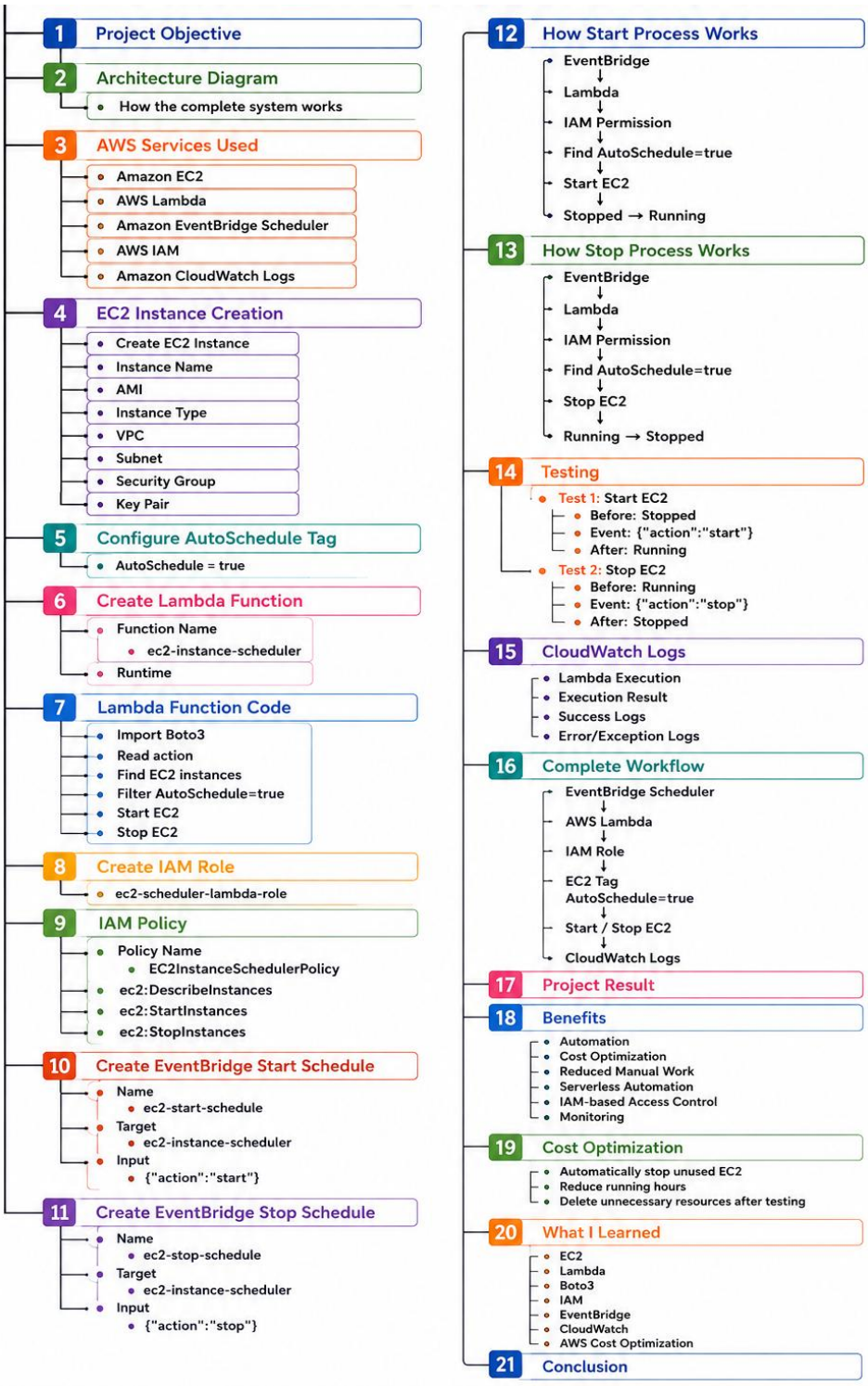
AWS EC2 Instance Schedule



AWS EC2 Instance Scheduler

Automated EC2 Start/Stop Using AWS Lambda and EventBridge Scheduler

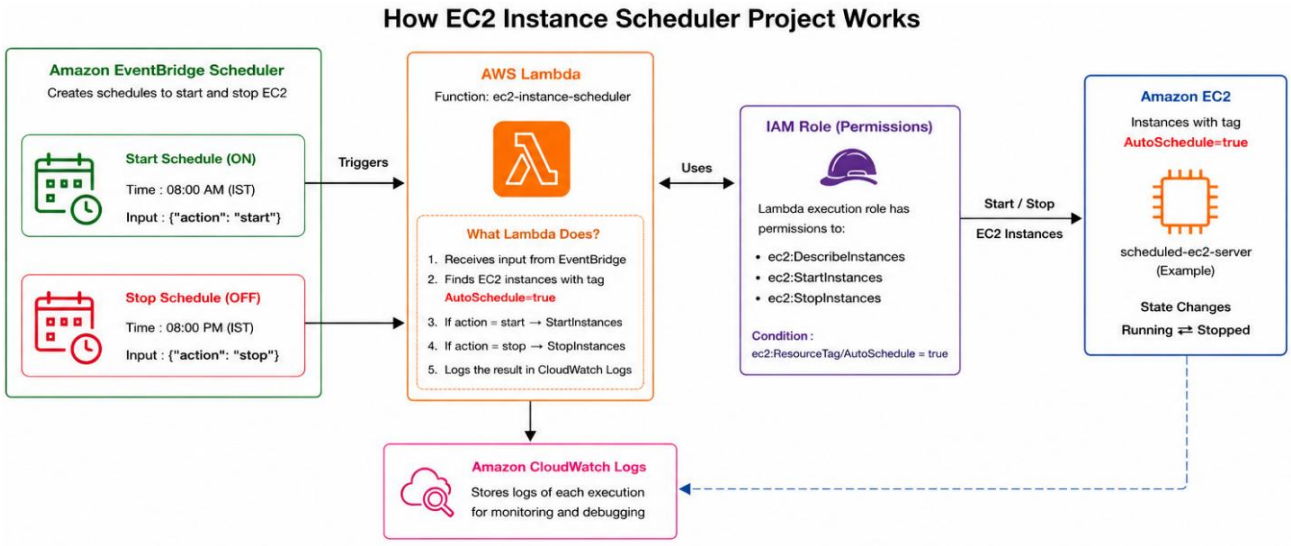
AWS services used :

- **Amazon**
- **AWS Lambda**
- **Amazon EventBridge**
- **CloudWatch Events scheduler**
- **AWS IAM**
- **CloudWatch Logs**

1. Project Objective :

The objective of this project is to automate the start and stop operations of Amazon EC2 instances using AWS Lambda and Amazon EventBridge Scheduler. EC2 instances are identified using the AutoSchedule=true tag, allowing only tagged instances to be managed by the automation.

2. Architecture Diagram



The architecture uses EventBridge Scheduler to trigger an AWS Lambda function at predefined times. Lambda uses an IAM execution role to identify EC2 instances with the AutoSchedule=true tag and start or stop them. Lambda execution results are recorded in Amazon CloudWatch Logs.

3. AWS Services Used :

AWS Service	Purpose
Amazon EC2	Instance that will be automatically started and stopped
AWS Lambda	Executes the EC2 start/stop automation
EventBridge Scheduler	Triggers Lambda at scheduled times
AWS IAM	Provides Lambda permission to access EC2
CloudWatch Logs	Stores Lambda execution logs

4. EC2 Instance Creation

4.1 Create EC2 Instance

- Instance name
- AMI
- Instance type
- VPC
- Subnet
- Security Group
- Key pair

Your instance name can be: scheduled-ec2-server

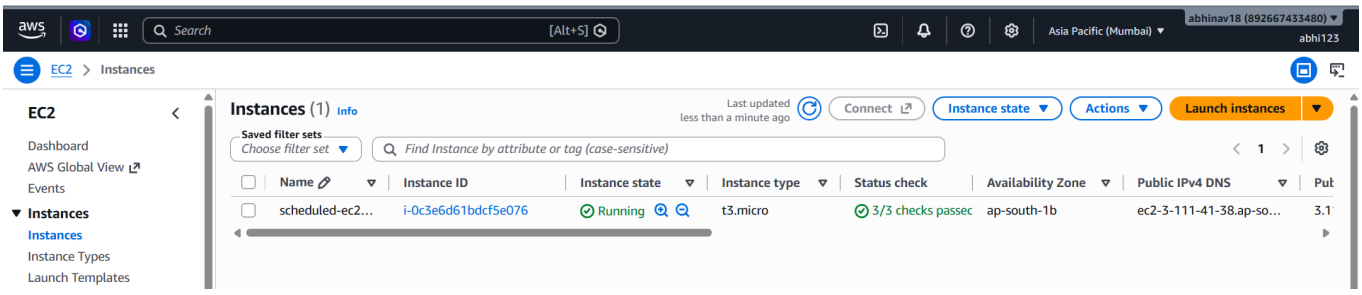


Figure 1: EC2 instance created for automated scheduling

5. Add AutoSchedule Tag :

Go to:

EC2 → Instances → Select Instance → Tags

The AutoSchedule=true tag identifies which EC2 instances should be managed by the Lambda automation. This allows the Lambda function to target specific instances instead of managing every EC2 instance in the account.

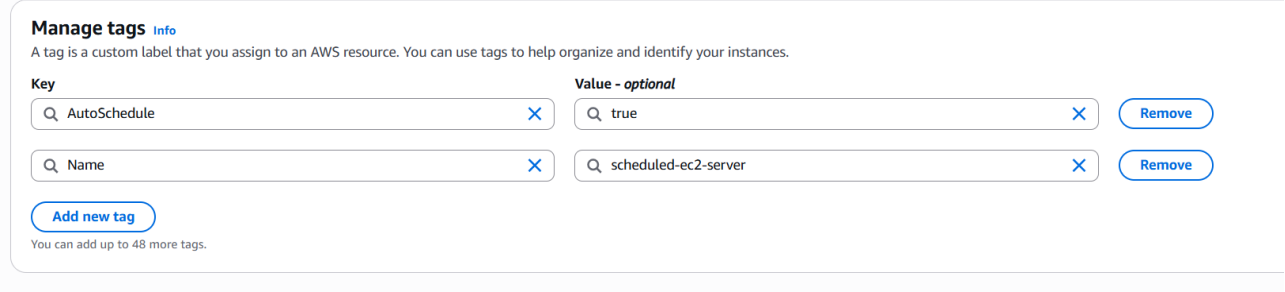


Figure 2: AutoSchedule=true tag configured on the EC2 instance

6. Create Lambda Function :

- **Lambda Function Name :** ec2-instance-scheduler
- **Runtime**
  Use the runtime you actually selected, for example:**Python**

An AWS Lambda function named ec2-instance-scheduler was created to automate EC2 instance start and stop operations.

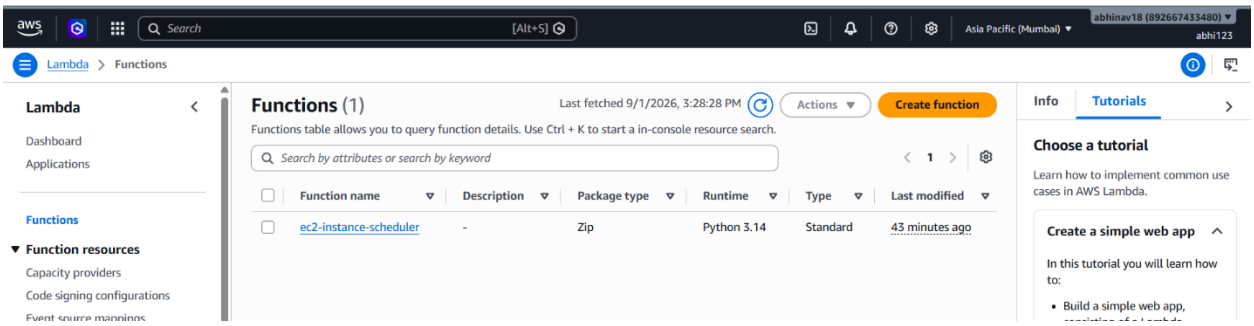


Figure 3: Lambda function created for EC2 scheduling

7. Lambda Function Code :

```
import boto3
ec2 = boto3.client('ec2')
def lambda_handler(event, context):
    action = event.get("action")
    response = ec2.describe_instances(
        Filters=[
            {
                'Name': 'tag:AutoSchedule',
                'Values': ['true']
            }
        ]
    )
    instance_ids = []
    for reservation in response['Reservations']:
        for instance in reservation['Instances']:
            instance_ids.append(instance['InstanceId'])
    if not instance_ids:
        return {"message": "No instances found"}

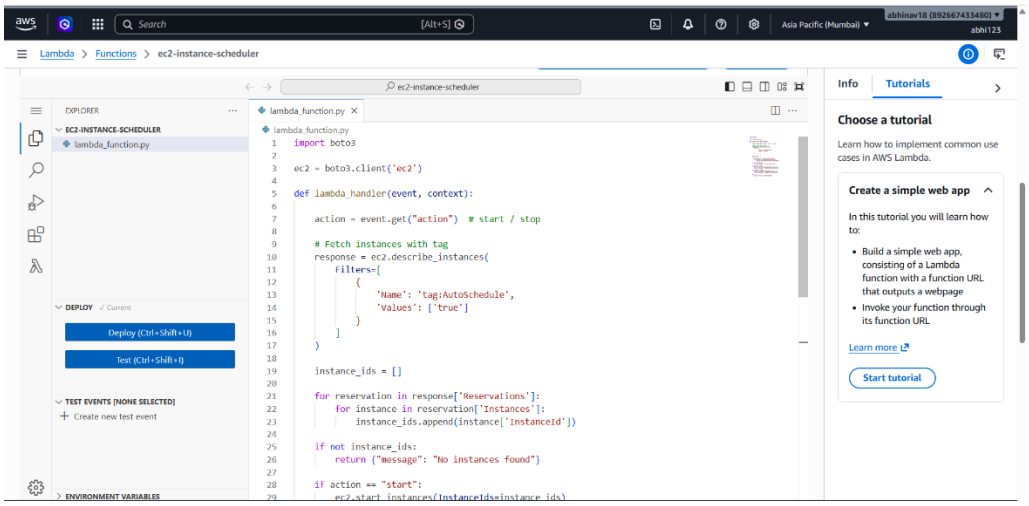
    if action == "start":
        ec2.start_instances(InstanceIds=instance_ids)
        return {"message": f"Started {instance_ids}"}

    elif action == "stop":
        ec2.stop_instances(InstanceIds=instance_ids)
        return {"message": f"Stopped {instance_ids}"}
    else:
        return {"error": "Invalid action"}
```

- **Lambda Logic :**

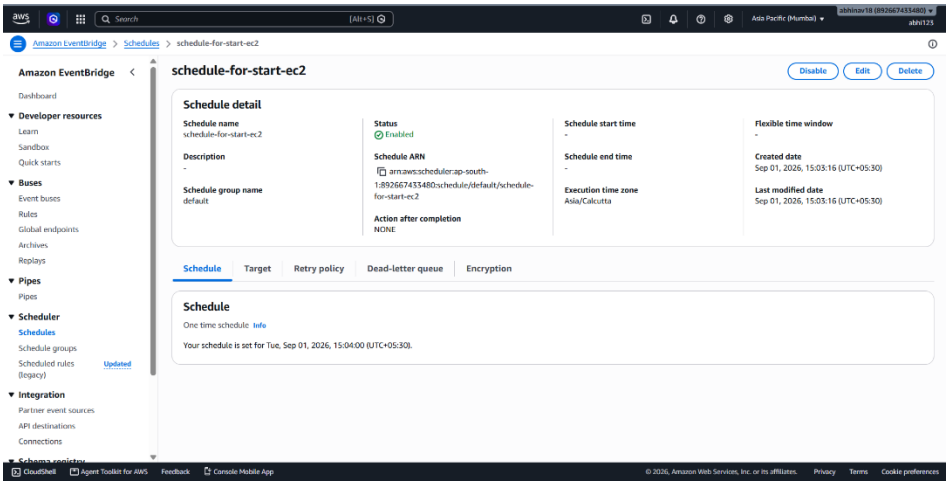
```
Receive action
    ↓
Find EC2 instances
    ↓
Filter AutoSchedule=true
    ↓
Get Instance ID
    ↓
Is action "start"?
   ↓ Yes
Start EC2

Is action "stop"?
   ↓ Yes
Stop EC2
```



8. IAM Role :

Recommended name: ec2-scheduler-lambda-role



Policy Name : EC2InstanceSchedulerPolicy

Permissions :

ec2:DescribeInstances

ec2:StartInstances

ec2:StopInstances

code for policy :

```
{
 "Version": "2012-10-17",
 "Statement": [
  {
   "Effect": "Allow",
```

```
      "Action": [
        "ec2:DescribeInstances"
      ],
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "ec2:StartInstances",
        "ec2:StopInstances"
      ],
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "ec2:ResourceTag/AutoSchedule": "true"
        }
      }
    }
  ]
}
```

Your policy also restricts Start/Stop permissions using:
AutoSchedule = true

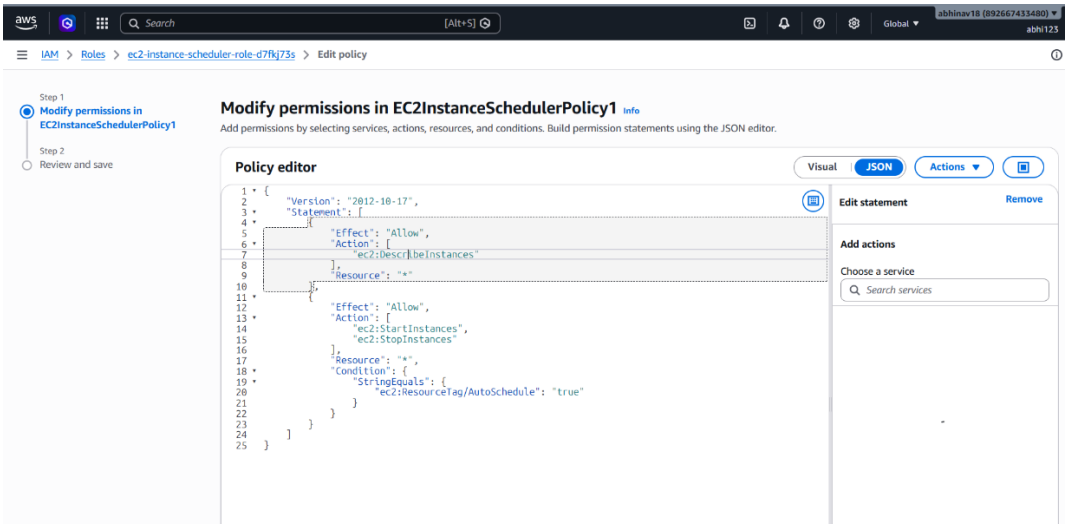


Figure 4: IAM policy allowing Lambda to manage EC2 instances with AutoSchedule=true

9. Create EventBridge Start Schedule :

Create: ec2-start-schedule

For testing, i used: 15:04

- **Input :**

```
{
 "action": "start"
}
```

- **Target :**

AWS Lambda

↓

ec2-instance-scheduler

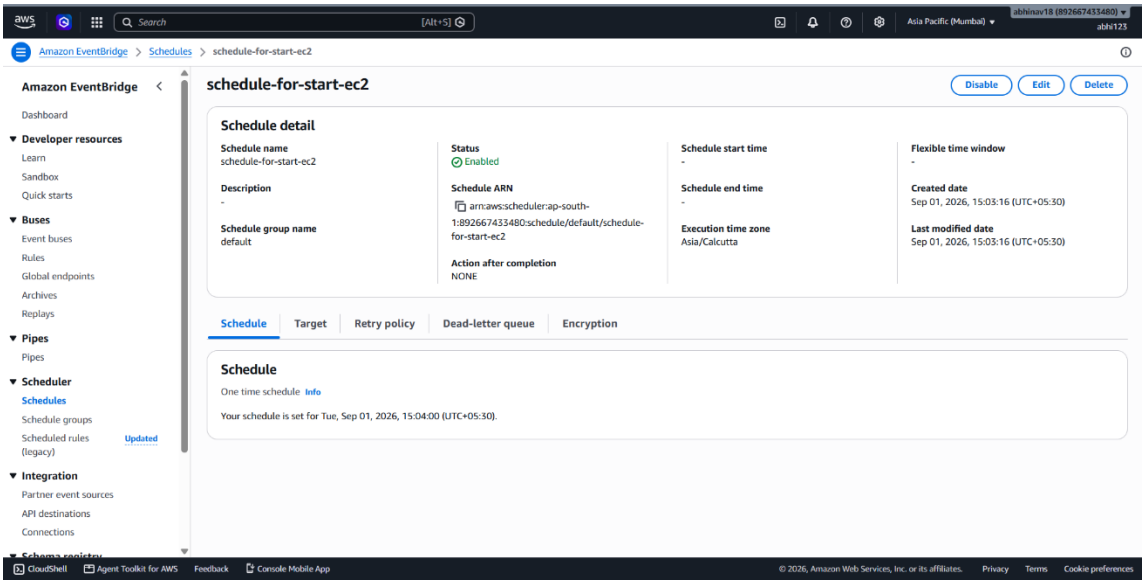


Figure 5: EventBridge schedule configured to trigger Lambda for EC2 start operation

10. Create EventBridge Stop Schedule :

Create: ec2-stop-schedule

- **Input :**

```
{
 "action": "stop"
}
```

- **Target :**

AWS Lambda

↓

ec2-instance-scheduler

For testing, i used: 15:06

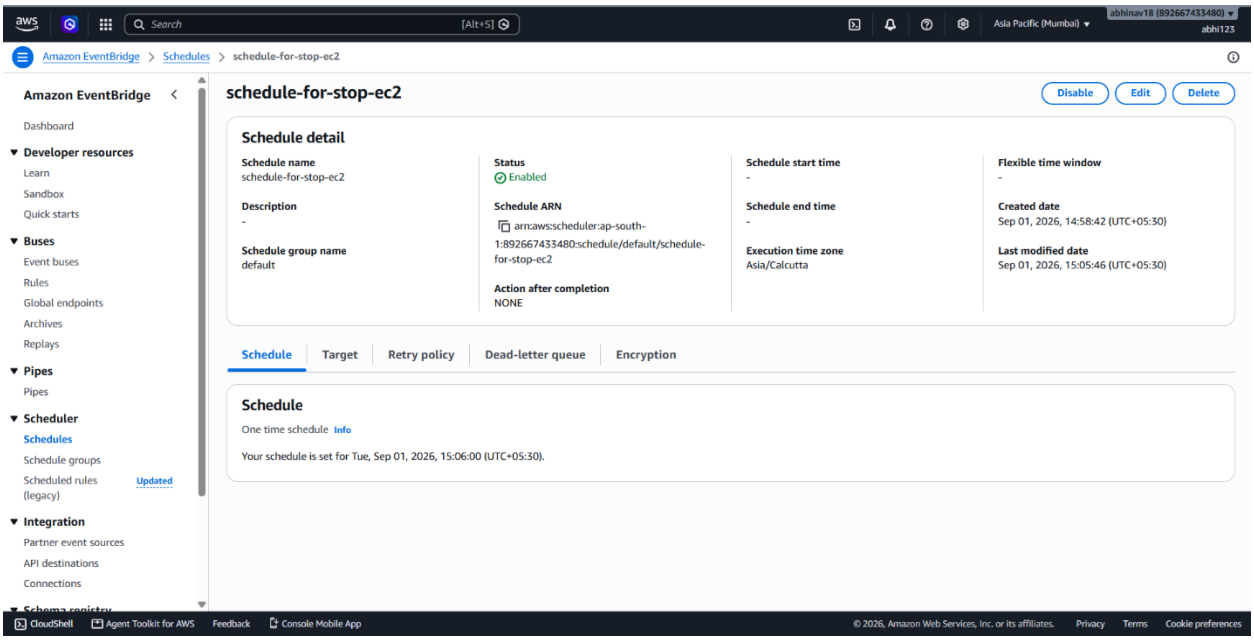


Figure 6: EventBridge schedule configured to trigger Lambda for EC2 stop operation

11. How the Start Process Works :

EventBridge

| {"action":"start"}

▼

Lambda

| Find AutoSchedule=true

▼

EC2

▼

Stopped → Running

At the configured start time, EventBridge Scheduler invokes the Lambda function and sends the {"action":"start"} event. Lambda searches for EC2 instances tagged AutoSchedule=true and calls the EC2 StartInstances API. The selected instance changes from the Stopped state to the Running state.

12. How the Stop Process Works

EventBridge

```
  │ {"action":"stop"}
  ▼
```

Lambda

```
  │ Find AutoSchedule=true
  ▼
```

EC2

```
  ▼
```

Running → Stopped

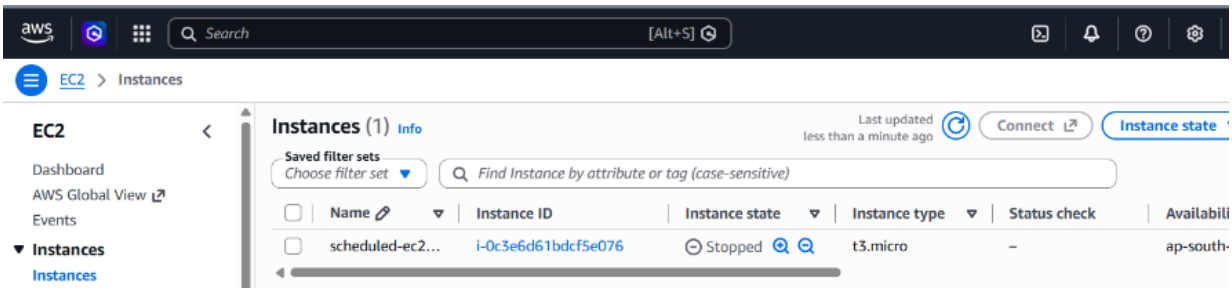
At the configured stop time, EventBridge Scheduler invokes the Lambda function with the {"action":"stop"} event. Lambda identifies the EC2 instances tagged AutoSchedule=true and calls the StopInstances API, changing the instance from Running to Stopped.

13. Testing

Test 1 — Start EC2 :

Before:

EC2 State: **Stopped**



Event:

```
{
 "action": "start"
}
```

After **:**

EC2 State: Running

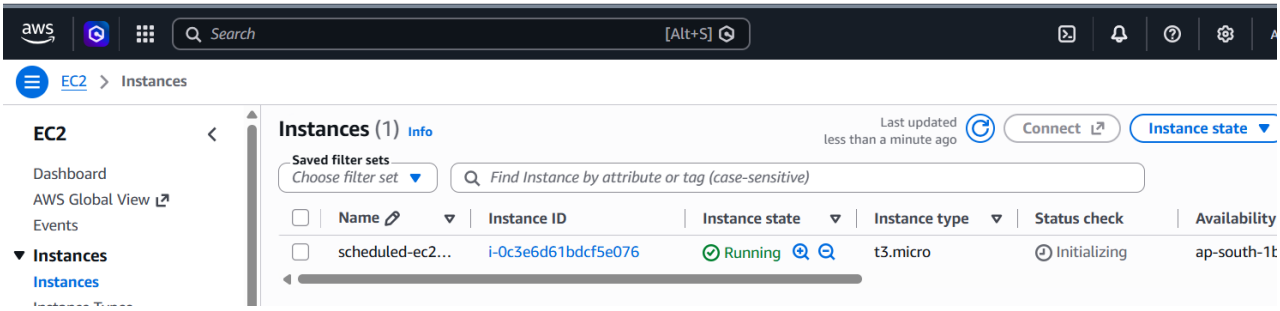
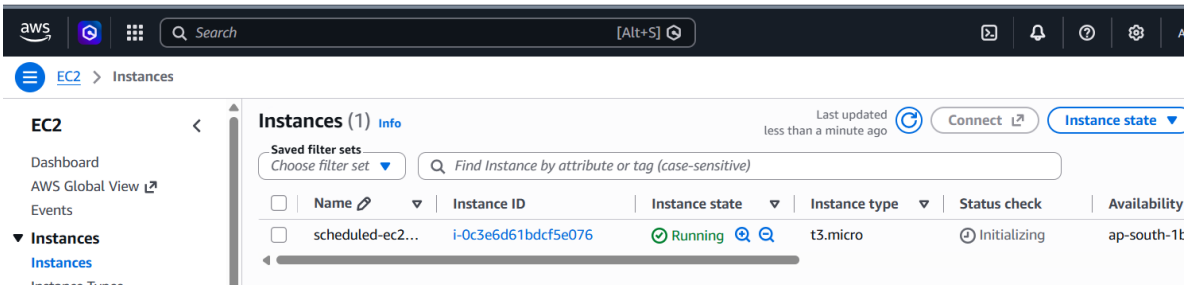


Figure 7: EC2 instance successfully started by the scheduled Lambda automation

Test 2 — Stop EC2

Before:

EC2 State: **Running**



Event:

```
{
 "action": "stop"
}
```

After:

EC2 State: **Stopped**

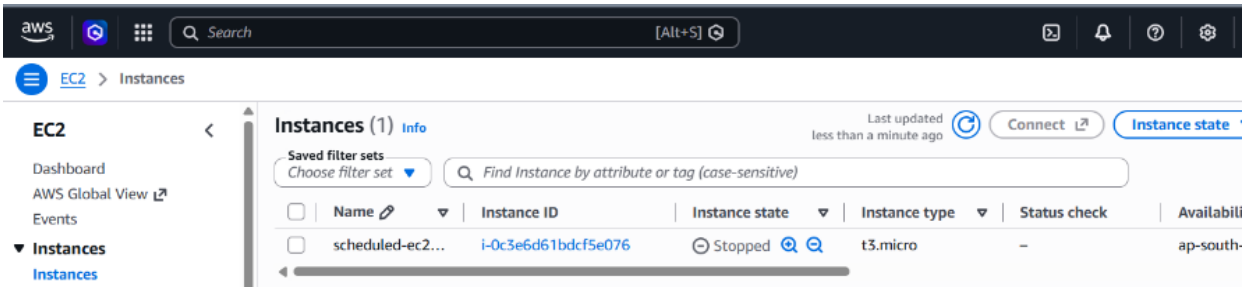
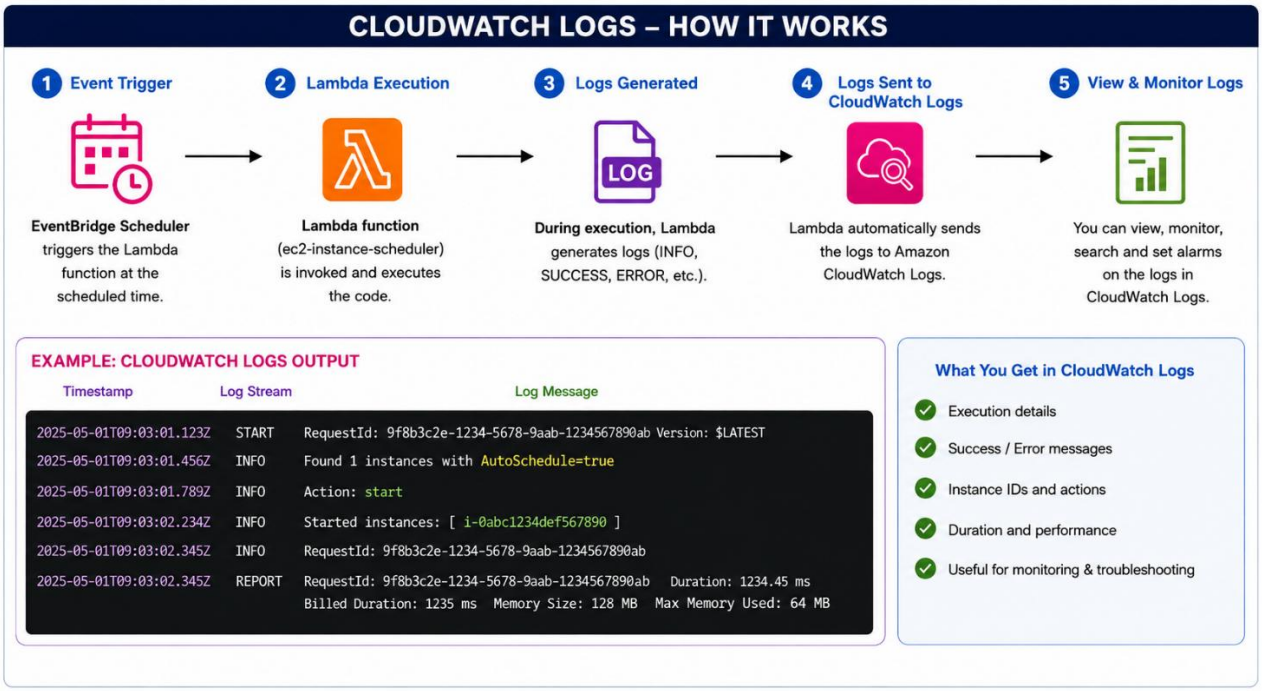
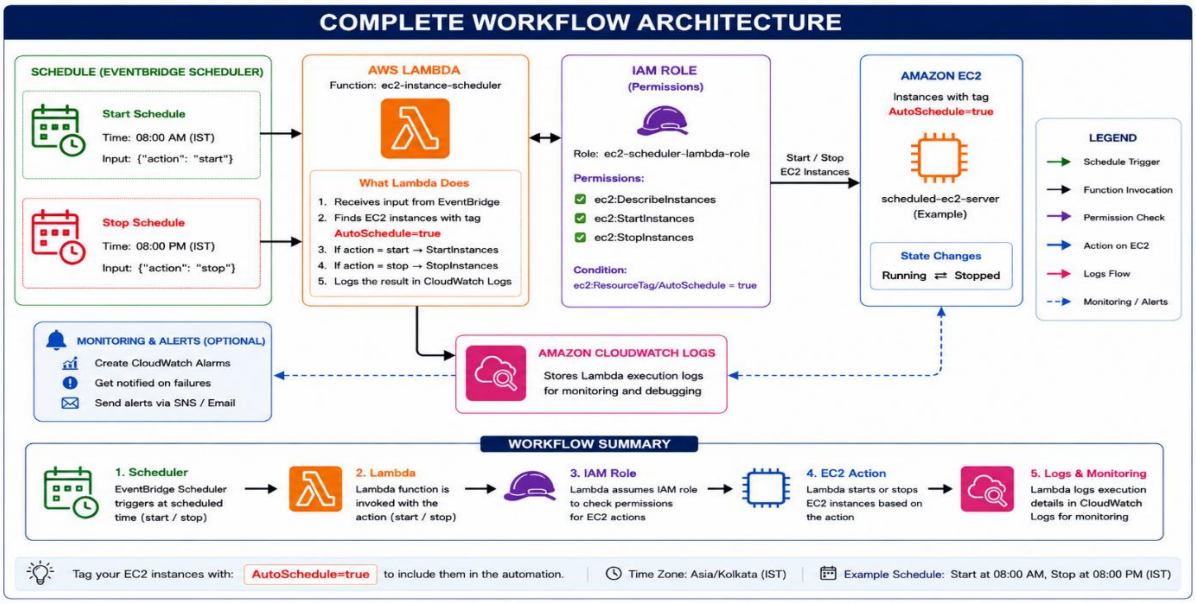


Figure 8: EC2 instance successfully stopped by the scheduled Lambda automation

14. CloudWatch Logs



15. Complete Workflow



16. Project Result

Result: The EC2 instance scheduling automation was successfully implemented using Amazon EventBridge Scheduler, AWS Lambda, IAM, and Amazon EC2. The Lambda function successfully identified EC2 instances using the AutoSchedule=true tag and performed the required start and stop operations according to the configured schedules.

17. Benefits

- Automates EC2 start and stop operations.
- Reduces unnecessary EC2 running time.
- Helps optimize AWS costs.
- Removes the need for manual instance management.
- Demonstrates serverless automation using Lambda.
- Uses IAM permissions to control EC2 access.
- Provides execution visibility through CloudWatch Logs.
- Can be extended to manage multiple tagged EC2 instances.
-

18. Cost Optimization

EC2 instances can be automatically stopped during periods when they are not required. This reduces unnecessary compute usage and helps optimize AWS costs.

And after finishing your AWS lab:

After testing and capturing the required evidence, unnecessary AWS resources were stopped or deleted to prevent additional charges.

19. What I Learned

Through this project, I learned how to:

1. Create and configure EC2 instances.
2. Use EC2 tags for resource selection.
3. Create AWS Lambda functions using Python and Boto3.
4. Configure IAM roles and permissions.
5. Create scheduled EventBridge rules.
6. Automate EC2 start and stop operations.
7. Monitor Lambda execution using CloudWatch Logs.

20. Conclusion

The AWS EC2 Instance Scheduler project was successfully implemented to automate the start and stop operations of an Amazon EC2 instance using AWS Lambda, Amazon EventBridge Scheduler, AWS IAM, and Amazon CloudWatch Logs. The EC2 instance was identified using the AutoSchedule=true tag, allowing Lambda to manage the required instance based on the scheduled action.

The project successfully demonstrated both EC2 start and stop automation, with the instance changing from Stopped → Running for the start operation and Running → Stopped for the stop operation. CloudWatch Logs provided visibility into Lambda execution, while IAM permissions controlled Lambda's access to EC2 resources.